

ATLAS OS

Product Requirements

Technical & requirements

Program	ATLAS Habitat Initiative
Product	ATLAS OS — the Habitat Twin (a.k.a. Smart admin)
First unit	ATLAS 01
Author	Elijah Royaie
Status	Draft v0.1 — living document
Companion docs	Business Plan · Legal & Compliance · Owner’s Project Requirements · Security & Data Compliance · Data Model · De-risking Plan · Blender Model Spec · RuView Presence · Cloud Automations

This is the PRD the README and Business Plan reference. It is the source of truth for **what ATLAS OS must do** and the acceptance criteria each capability is judged against.

1. Purpose & scope

ATLAS OS is the human window into a self-sufficient habitat: an interactive **3D digital twin optimization layer**, over live per-floor telemetry, incident triage, and resident broadcast. **It is never a dependency** — life safety is held by safety-rated building controls (BMS/PLC), not by this app or any cloud service.

In scope: the twin, per-floor telemetry, incident feed, broadcast, KPI/Resilience strip, fleet rollup, sensor ingestion, building simulator, camera-free presence sensing, the analytics/anomaly engine, the AI ops copilot, the documents library + legal pages, the public marketing/stakeholder pages, and the gITF building model pipeline (per-floor modules + optional whole-building heatmap).

Out of scope: direct actuation of life safety systems, financial/billing, resident identity/auth beyond read views, native mobile apps.

2. Personas

Persona	Needs
Operator / building manager	At-a-glance health, fast incident triage, broadcast to residents
Resident	Trust signals: autonomy %, food/water/energy status, clear incident comms
Investor / donor	A legible, demoable proof the thesis is real (simulator + twin)
Engineer / SRE	Sensor ingestion, data integrity, uptime, auditability

3. Functional requirements

Feature labels mirror the code ([F1](#)–[F12](#)); each row is testable.

ID	Requirement	Acceptance criteria
F1	3D Habitat Twin	Orbitable tower; one floor per need, color coded; floors with an active incident pulse; selecting a floor opens its telemetry
F2	Per floor telemetry	Tapping a floor shows live energy/ water/food/occupancy/climate; metrics are sparse tolerant (only what a floor reports)
F3	Security & incident feed	Severity ranked (crit/warn/info), live, "d15 s poll; clicking an event focuses its floor
F4	Resident broadcast	Compose + send; a broadcast mirrors an info incident into the feed; POST validates body, returns 201/400
F5	Building KPI / Resilience strip	Resilience Index (blended energy autonomy / non potable reuse / food amenity sub scores, 0–100), battery, solar, dwellings, residents, open incidents — derived by aggregating floors. KPI labels are honest by design (grid tied, not off grid; non potable reuse only; food = amenity)
F7	Fleet view	Multi building rollup, separate from the single building console
F11	Sensor ingestion	Tagged sensor points (Brick/ Haystack style); ingest + query via / api/sensors; 201/400 contract
F12	Building Simulator	Architectural operating twin by default: cut away floors, curtain-wall frame, detailed floor GLB modules, working elevator(s), switchback stairs, rooftop solar, residents; Pixel and ASCII modes remain opt in retro fallbacks
+	Guided walk through	Animated rooftop!'underground camera descent with annotated callouts
+	WebXR AR	"Enter AR" on WebXR capable devices; graceful fallback to walk through where unsupported
F13	Presence sensing	Camera free occupancy/ person count via RuView ESP32 nodes (sys.presence), /api/ presence; seed fallback when hardware absent. See RuView Presence

F14	Analytics & anomaly engine	Pure, deterministic trend/anomaly detection + building health score over telemetry; AnalyticsPanel + /api/analytics. Observe and advise only (OPR §4)
F15	AI ops copilot	LLM agnostic ops Q&A over a compact building snapshot, with a deterministic local fallback (works with no API key / offline). /api/copilot. Never issues control actions
F16	Documents library	/documents in page PDF reader over the business/technical/legal papers in docs/; each downloadable as PDF (generated by scripts/generate-doc-pdfs.ts)
F17	Legal pages	/legal + /legal/privacy, /legal/terms, /legal/dpa rendered as first class HTML routes from the same markdown sources
F18	Marketing & stakeholder pages	/landing, /for + per audience pages (developers, operators, residents, investors, cities, architects, sustainability), /portfolio, /brand
F19	3D model pipeline (glTF)	Per floor floor-<need>.glb modules (built by _blender/build_atlas_floors.py, ~370 KB total) loaded additively over the simulator; optional whole building hero model (atlas-01.glb) behind a Hero toggle with graceful procedural fallback. Authoring/binding contract: Blender Model Spec
F20	SEO surface	Request time canonical base URL; sitemap.xml, robots.txt, and a human /sitemap all derived from one SITE_ROUTES registry

4. Non functional requirements

Area	Requirement
Local first	App renders fully on in memory seed data with no database connected; DATABASE_URL enables optional Neon Postgres persistence
Performance	Twin interactive at 60 fps target on a modern laptop; live model "d 60k tris / "d 2.5 MB; Pixel mode remains available as the lightweight fallback
Responsiveness	Mobile first; verified on Desktop Chrome, Tablet (Galaxy Tab S9), Mobile (Pixel 5) Playwright projects; safe area insets honored
Accessibility	WCAG 2.1 AA targets for DOM chrome; keyboard navigable controls; sufficient contrast in dark ops palette

Internationalization	UI strings translatable; per user preference, multilingual + dark mode variants maintained in parallel
Reliability	Cloud app is optional; degrades to seed/offline; target 99.9% uptime for hosted instances
Security	See Security & Data Compliance; no PII in seed data; least privilege API
Auditability	Incidents/broadcasts logged; sensor ingestion timestamped; CRA/SR&ED compatible R&D logs retained
Privacy by design	Presence sensing is camera free (ESP32 presence nodes); data minimization (see Privacy Policy)

5. Data model (domain types)

`Floor`, `Incident`, `Broadcast`, `BuildingKpis`, `SensorPoint`, `Building` (`src/lib/types.ts`). Enums for colors/labels in `src/lib/ui.ts` (kept in sync with `tailwind.config.ts`). `FloorMetrics` is all optional; KPIs aggregate across floors. `OccupancyClassification` (model adds `ModelAddyGroup`, `UseScope`) and `Resilience Index` sub scores, plus the dwellings/beds/floors distinction — full the `ATLAS Data Model`. The right sizing rationale behind these choices is recorded in the [Data Model](#).

6. APIs

`/api/floors`, `/api/incidents`, `/api/broadcasts`, `/api/sensors`, `/api/presence`, `/api/analytics`, `/api/copilot`, `/api/brand/zip` — all force-dynamic, validate POST 201 create / 400 bad input). They delegate to the JSN optional lib `analytics` and `copilot` and **observe and advise only** (OPR §4) — they never actuate anything.

7. Architecture constraints

Next.js 14 (App Router) + React 18 + TS strict; three.js via R3F/drei; Tailwind; Drizzle ORM over Neon. The **safety boundary is absolute**: ATLAS OS observes and advises; it does not control fire, egress, electrical isolation, or biogas systems — those are deterministic, [OPR](#) safety rated, and independent of cloud/AI (see [Data Model](#)).

8. Testing & release gates

`npm run verify` = typecheck + lint + build + Playwright matrix. CI ([.github/workflows/ci.yml](#)) runs build + e2e on every push to `main`. A change ships only when these are green.

9. Out of scope / future

Native apps, resident auth & billing, direct actuation, multi tenant org management, on device ML — tracked separately, not P0.

Living document — update alongside the code it specifies.

